LLD Phase 5 — PDF 4

Snake and Ladder · Cache with TTL · Meeting Room Scheduler · Chess | Requirements first, hints in the middle, full solutions at the back

> **Colour key. Purple = the interviewer's words · Blue = a note or hint · Red = a common mistake · Green = a rule to remember · Amber = how to say it in an interview.**

Contents

Before you start — the loop · the rush helper · time boxes · warm-up: two PDF 3 designs from memory, then 12 quick ones

Problem 1 — Snake and Ladder (small)

Problem 2 — Cache with TTL and eviction (core)

Problem 3 — Meeting Room Scheduler (core)

Problem 4 — Chess (heavy)

Hints — three levels per problem: a nudge, the shape, the key decision

Solutions 1–4 — clarifying questions · Part A model answer · what to write first · full code · Part C answers · common mistakes · 10-point checklist

Back pages — warm-up answers · mistake log · re-do schedule

Before you start

Same format as PDF 3. Each problem opens with the interviewer's words, in purple; write down your own questions before reading the requirements under it. The solutions are interview-sized: plain Java, only the checks the requirements ask for, and a "write this first" order for when time runs short. The loop stays the same: Part A on paper, one hint level only after 15 minutes stuck, Part B until the output matches exactly, Part C out loud, then the solution and its checklist.

New in PDF 4	What it means for you
Time you can control	The cache takes its time from a `Clock` passed in from outside, so a test can move time forward without waiting. Expect this question in any problem with expiry or timeouts.
Rules as objects	In chess, every piece knows its own moves, so the game asks the piece instead of switching on its type. That is polymorphism doing real work.
Intervals	Meeting rooms are about time slots that must not overlap. One `TreeMap` lookup answers "does this clash?" in O(log n).

The rush helper (given, same as PDF 3)

```
// Given: runs task(0) .. task(n-1) on n threads that all start together; counts the trues.
static int rush(int n, IntPredicate task) throws Exception {
    ExecutorService pool = Executors.newFixedThreadPool(n);
    CountDownLatch go = new CountDownLatch(1);
    List<Future<Boolean>> results = new ArrayList<>();
    for (int i = 0; i < n; i++) {
        int id = i;
        results.add(pool.submit(() -> { go.await(); return task.test(id); }));
    }
    go.countDown();
    int trues = 0;
    for (Future<Boolean> f : results) if (f.get()) trues++;
    pool.shutdown();
    return trues;
}
```

Used by the Meeting Room Scheduler's RUSH command. It needs `import java.util.concurrent.*;` and `import java.util.function.IntPredicate;`, and `main` must declare `throws Exception`.

Problem	Size	Time box	Format
1. Snake and Ladder	small	45 min	10 design + 35 code
2. Cache with TTL	core	75 min	20 design + 55 code
3. Meeting Room Scheduler	core	75 min	20 design + 55 code
4. Chess	heavy	100 min	25 design + 75 code

Warm-up (15 minutes)

Part 1 — from memory. Don't open PDF 3. Write on paper, then check against the back pages.

W1	**Rate Limiter** (5 min). For fixed window, sliding log and token bucket: what one client's state holds, and one weakness or strength of each. Where does the lock go, and why does `computeIfAbsent` matter? Where in the PDF 3 output did the fixed window let a burst through?
W2	**BookMyShow** (5 min). Which seat facts belong to the screen and which to the show? What is the one indivisible step in a hold, and what do you lock? Draw the booking's states. Who releases an expired hold?

Part 2 — name the pattern (5 min). All from one app: the Pune Metro app. Name the pattern, or say "nothing" and what you'd use instead.

1. Fares depend on distance slabs, with discounts for students and senior citizens. The rules change every year.
2. A metro card goes ISSUED → ACTIVE → BLOCKED (reported lost) → CLOSED. What top-up, tap-in and refund do depends on the status.
3. The gates come from a vendor whose library offers `gateOpen(int code)`. Our code calls `Gate.allow(Card)` everywhere.
4. When a train is delayed: update the station displays, notify riders on that line, log it for reports. More reactions will come.
5. Refunds: up to Rs 100 are automatic, up to Rs 1,000 the station master approves, above that the head office.
6. Buying a QR ticket: pick stations, work out the fare, take a UPI payment, make the QR, send an SMS. The app, the kiosk and the WhatsApp bot all do this.
7. Lines have stations, which have platforms. The dashboard wants the day's footfall at any level.
8. Cards not used for a year are deactivated.
9. A journey-planner request has a start and an end (required) and optional settings: departure time, fewest changes, step-free access. It never changes once made.
10. Every tap must be written to an audit log, and refunds need a supervisor's role, without changing the card service's code.
11. A station has a code and a name.
12. "Repeat yesterday's trip" copies a saved journey so it can be edited.

Problem 1 — Snake and Ladder

Small · 45 minutes (10 design + 35 code) · machine-coding format

> **The interviewer says:** "Design Snake and Ladder. A board with snakes and ladders, two or more players taking turns with one dice. Land at the bottom of a ladder and you climb it; land on a snake's head and you slide down. The first to reach the last cell wins."

After your questions, the requirements are

R1	The board has cells 1 to N. Players start at 0, off the board.
R2	A snake goes down; a ladder goes up. Both ends must be on the board, nothing can start at N, and each cell can be the start of at most one snake or ladder. An end may land on the start of another (the player keeps following), but a set-up that would make a loop is refused. Checked in this order: both ends on the board, the direction, starting at N, the start cell already used, the loop.
R3	Players join before the game starts, with unique names. A game needs at least 2 players. Turns go in the order they joined.
R4	The dice value comes in the input (1 to 6). The player moves forward by it; if that would go past N, they stay where they are. After moving, they follow snakes and ladders until they land on a plain cell.
R5	Rolling a six gives the same player another turn, unless they've just won.
R6	Reaching N exactly wins, and the game is over.
NF1	The dice must be replaceable: random in the real game, fixed values in tests, two dice later.

Out of scope: three sixes in a row losing the turn, saving games, a computer player.

Commands and exact output

Command	Output
`BOARD n`	`Board of 30 cells` (a new board and a new game)
`SNAKE head tail`	`Snake 27 -> 1`
`LADDER bottom top`	`Ladder 3 -> 12`
`PLAYER name`	`Player Asha joined`
`START`	`Game started: Asha, Ravi`
`ROLL n`	`Asha rolled 3: 0 -> 3, ladder to 12, ladder to 22` · `Asha rolled 5: 22 -> 27, snake to 1` · `Asha rolled 5: stays at 28 (needs exactly 2)` · ends with `, rolls again` after a six, or `. Ravi wins!`
`POSITIONS`	`Asha 28, Ravi 30`

Error	Output
`an end off the board / wrong direction`	`Cell 31 is off the board` · `A snake must go down` · `A ladder must go up`
`starting at N / start cell used / a loop`	`Nothing can start at 30` · `Cell 3 already has a snake or ladder` · `That would make a loop`
`joining late / a name taken / too few players`	`Game already started` · `Player Asha already exists` · `Need at least 2 players`
`rolling before START / after a win / a bad value`	`Game not started` · `Game is over` · `Roll must be 1 to 6`

Part A — design

1	Entities: board, cell, snake, ladder, player, dice, game. Which deserve a class, and which are just data? (Do snakes and ladders need two classes?)
2	Who owns "where does cell 27 send me?", and who owns "whose turn is it?"
3	How do you detect that a new snake or ladder would make a loop?
4	NF1: how does the dice become replaceable?

Part B — code

input.txt

```
BOARD 30
LADDER 3 12
LADDER 12 22
SNAKE 27 1
SNAKE 17 6
SNAKE 24 31
SNAKE 9 14
LADDER 25 30
SNAKE 30 2
LADDER 3 8
SNAKE 22 3
LADDER 6 17
PLAYER Asha
START
PLAYER Ravi
PLAYER Asha
ROLL 3
START
ROLL 3
ROLL 7
ROLL 6
ROLL 5
ROLL 5
ROLL 6
ROLL 4
ROLL 2
ROLL 3
ROLL 6
ROLL 5
ROLL 6
ROLL 6
POSITIONS
ROLL 1
```

expected output

```
Board of 30 cells
Ladder 3 -> 12
Ladder 12 -> 22
Snake 27 -> 1
Snake 17 -> 6
Cell 31 is off the board
A snake must go down
Ladder 25 -> 30
Nothing can start at 30
Cell 3 already has a snake or ladder
That would make a loop
That would make a loop
Player Asha joined
Need at least 2 players
Player Ravi joined
Player Asha already exists
Game not started
Game started: Asha, Ravi
Asha rolled 3: 0 -> 3, ladder to 12, ladder to 22
Roll must be 1 to 6
Ravi rolled 6: 0 -> 6, rolls again
Ravi rolled 5: 6 -> 11
Asha rolled 5: 22 -> 27, snake to 1
Ravi rolled 6: 11 -> 17, snake to 6, rolls again
Ravi rolled 4: 6 -> 10
Asha rolled 2: 1 -> 3, ladder to 12, ladder to 22
Ravi rolled 3: 10 -> 13
Asha rolled 6: 22 -> 28, rolls again
Asha rolled 5: stays at 28 (needs exactly 2)
Ravi rolled 6: 13 -> 19, rolls again
Ravi rolled 6: 19 -> 25, ladder to 30. Ravi wins!
Asha 28, Ravi 30
Game is over
```

Part C — pressure questions

1	**Change.** "Three sixes in a row: the player loses the turn and goes back to where the turn began."
2	**Change.** "Add special cells: a 'skip next turn' cell and a 'roll again' cell."
3	**Testing.** "How do you test a game that depends on a random dice?"
4	**Scale.** "Make it an online game: thousands of games at once, players on phones."
5	**Defend it.** "Why not classes Snake and Ladder, both extending Jump?"

Problem 2 — Cache with TTL and eviction

Core · 75 minutes (20 design + 55 code) · machine-coding format

> **The interviewer says:** "Design an in-memory key-value cache where each entry can have a time-to-live. An expired entry must never be returned. The cache has a size limit, so when it's full, something has to be evicted. Many threads use it, and we need to test it without waiting for real time to pass."

After your questions, the requirements are

R1	A cache has a capacity (at least 1). `put(key, value, ttl)` : ttl in seconds; 0 means it never expires; negative is refused. Putting an existing key replaces its value and its TTL.
R2	An entry put at time t with ttl s expires when the clock reaches t + s. An expired entry is never returned: reading it removes it, and it counts as a miss.
R3	Every read of a key (GET or TTL) counts as a use of it.
R4	When the cache is full and a **new** key arrives: if any entry has expired, remove the one that expired first; otherwise evict the least recently used.
R5	Cleanup removes every expired entry, in order of expiry. This is what a background job would run every few seconds.
R6	Stats: size, hits and misses (from GET), expired (entries removed because they expired, however they were found) and evictions (least-recently-used ones only).
R7	Time comes from a clock given to the cache. TIME moves it forward; it can't go back.
NF1	Tests must control time; the cache never reads the system clock directly.
NF2	Safe to use from many threads.

Allowed this time: `LinkedHashMap` in access order (you built the list yourself in PDF 3). **Out of scope:** sizes in bytes, saving to disk, sharing across servers.

Commands and exact output

Command	Output
`CREATE capacity`	`Cache ready, capacity 3`
`TIME t`	`Time 26`
`PUT key value ttl`	`Put a, expires at 5` · `Put c, no expiry` · with room made: `Put f, expires at 29, evicted d (expired)` or `…, evicted b (least recent)`
`GET key`	`a = 1` or `a: miss`
`TTL key`	`b: 4s left` · `c: no expiry` · `a: miss`
`CLEANUP`	`Cleanup removed 2: g, h` or `Cleanup removed nothing`
`STATS`	`Size 1 of 3, hits 2, misses 3, expired 5, evictions 2`
`(errors)`	`Capacity must be at least 1` · `TTL cannot be negative` · `Time cannot go back`

Part A — design

1	Expired entries can be found two ways: when someone reads them, or by a regular sweep. What does each one cost, and why do real caches do both?
2	R4 needs two orders at once: by last use, and by expiry time. Which structures give you each?
3	NF1: how does the cache get the time? What does a test pass in, and what does production pass in?
4	Where does the lock go? Who calls `cleanup()` in production, and on which thread?

Part B — code

input.txt

```
CREATE 3
PUT a 1 5
PUT b 2 10
PUT c 3 0
GET a
TIME 6
GET a
TTL b
TTL c
PUT d 4 20
PUT e 5 0
GET b
PUT x 9 -5
TIME 26
PUT f 6 3
CLEANUP
TIME 40
CLEANUP
PUT g 7 2
PUT h 8 3
TTL g
TIME 30
TIME 50
CLEANUP
GET c
GET e
STATS
CREATE 0
```

expected output

```
Cache ready, capacity 3
Put a, expires at 5
Put b, expires at 10
Put c, no expiry
a = 1
Time 6
a: miss
b: 4s left
c: no expiry
Put d, expires at 26
Put e, no expiry, evicted b (least recent)
b: miss
TTL cannot be negative
Time 26
Put f, expires at 29, evicted d (expired)
Cleanup removed nothing
Time 40
Cleanup removed 1: f
Put g, expires at 42
Put h, expires at 43, evicted c (least recent)
g: 2s left
Time cannot go back
Time 50
Cleanup removed 2: g, h
c: miss
e = 5
Size 1 of 3, hits 2, misses 3, expired 5, evictions 2
Capacity must be at least 1
```

> **Three lines to check by hand.** At t=6, `a` (expired at 5) is a miss and is removed, so `PUT d` needs no eviction. `PUT e` then finds no expired entry and evicts `b`, the least recently used. At t=26, `d` expires exactly then (26 ≤ 26), so it goes before any live entry.

Part C — pressure questions

1	**Change.** "Run the cleanup automatically every second in production." Show it.
2	**Concurrency.** "With one lock, the cleanup blocks every reader while it runs. With a million entries, that hurts." What then?
3	**Change.** "Some keys should refresh their TTL every time they're read (sliding expiry)."
4	**Failure.** "A popular key expires, and 500 requests miss at the same moment and all hit the database."
5	**Defend it.** "Why a Clock interface? Just call `System.currentTimeMillis()`."

Problem 3 — Meeting Room Scheduler

Core · 75 minutes (20 design + 55 code) · machine-coding format, with a concurrency test

> **The interviewer says:** "Design a meeting room booking system for our office. Rooms have different capacities. People book a room for a time slot, and a room must never be double-booked. Sometimes people just want any free room for N people. They can cancel, and they want to see a room's schedule."

After your questions, the requirements are

R1	Rooms have unique names and a number of seats.
R2	Times are HH:MM on one day. A meeting occupies [start, end): one ending at 11:00 doesn't clash with one starting at 11:00. The end must be after the start.
R3	Booking a named room is refused, checked in this order, if: a time is badly written, the room is unknown, the end isn't after the start, the meeting has more people than seats, or it clashes with a meeting already there (the error names that meeting).
R4	Booking any room picks the **smallest** room that fits and is free; a tie goes to the name in alphabetical order. If none, it's refused.
R5	Listing free rooms for a slot and a group size uses the same order.
R6	Meetings get IDs M1, M2, ... (only successful bookings take a number). Cancelling by ID frees the slot. A room's schedule lists its meetings by start time.
NF1	Many people book at the same moment: one room and slot never goes to two meetings (RUSH tests this). Bookings in different rooms must not wait for each other.
NF2	The rule for choosing "any room" will change (nearest floor, has a projector).

Out of scope: several days, recurring meetings and invitations (Part C asks), time zones, buildings.

Commands and exact output

Command	Output
`ROOM name seats`	`Room Ganga added, seats 6`
`BOOK room start end organiser people`	`M1: Ganga 10:00-11:00, asha, 5 people`
`BOOK_ANY start end organiser people`	same format, for the room chosen
`FREE start end people`	`Free: Godavari (4), Yamuna (10)` or `No room free`
`CANCEL id`	`Cancelled M1`
`SCHEDULE room`	one line per meeting: `09:00-10:00 M3 meera (2)`; or `Godavari is free all day`
`RUSH room start end n`	n requests for that room and slot at the same moment: `Rush: 1 booked, 49 refused`

Error	Output
`a name taken / unknown room / unknown meeting`	`Room Ganga already exists` · `Unknown room Kaveri` · `Unknown meeting M1`
`bad time / end not after start / too many people`	`Invalid time 25:00` · `End must be after start` · `Ganga seats only 6`
`a clash / no room for BOOK_ANY`	`Ganga is busy 10:00-11:00 (M1)` · `No room free for 3 people at 10:30-11:00`

Part A — design

1	Where do a room's meetings live, and in what structure, so that "does this slot clash?" is fast?
2	Two meetings [s1, e1) and [s2, e2) clash exactly when…? Write the condition.
3	NF1: what is the one indivisible step in a booking, and what do you lock? How does BOOK_ANY stay safe without locking every room?
4	NF2: where does the "which room first" rule live?

Part B — code

input.txt

```
ROOM Ganga 6
ROOM Godavari 4
ROOM Yamuna 10
ROOM Ganga 8
BOOK Ganga 10:00 11:00 asha 5
BOOK Ganga 10:30 11:30 ravi 3
BOOK Ganga 11:00 12:00 ravi 3
BOOK Ganga 09:00 10:00 meera 2
BOOK Ganga 12:00 11:00 meera 2
BOOK Ganga 13:00 14:00 kiran 7
BOOK Kaveri 10:00 11:00 kiran 2
BOOK Ganga 25:00 26:00 kiran 2
FREE 10:00 11:00 3
BOOK_ANY 10:00 11:00 neha 3
BOOK_ANY 10:30 11:00 arjun 3
BOOK_ANY 10:30 11:00 isha 3
BOOK_ANY 10:30 11:00 dev 12
SCHEDULE Ganga
SCHEDULE Godavari
CANCEL M1
CANCEL M1
FREE 10:00 11:00 5
SCHEDULE Ganga
RUSH Godavari 14:00 15:00 50
SCHEDULE Godavari
```

expected output

```
Room Ganga added, seats 6
Room Godavari added, seats 4
Room Yamuna added, seats 10
Room Ganga already exists
M1: Ganga 10:00-11:00, asha, 5 people
Ganga is busy 10:00-11:00 (M1)
M2: Ganga 11:00-12:00, ravi, 3 people
M3: Ganga 09:00-10:00, meera, 2 people
End must be after start
Ganga seats only 6
Unknown room Kaveri
Invalid time 25:00
Free: Godavari (4), Yamuna (10)
M4: Godavari 10:00-11:00, neha, 3 people
M5: Yamuna 10:30-11:00, arjun, 3 people
No room free for 3 people at 10:30-11:00
No room free for 12 people at 10:30-11:00
09:00-10:00 M3 meera (2)
10:00-11:00 M1 asha (5)
11:00-12:00 M2 ravi (3)
10:00-11:00 M4 neha (3)
Cancelled M1
Unknown meeting M1
Free: Ganga (6)
09:00-10:00 M3 meera (2)
11:00-12:00 M2 ravi (3)
Rush: 1 booked, 49 refused
10:00-11:00 M4 neha (3)
14:00-15:00 M6 team (1)
```

Part C — pressure questions

1	**Change.** "Weekly recurring meetings: every Monday 10:00–11:00 for 10 weeks."
2	**Change.** "Invite attendees, and email them when a meeting is booked or cancelled."
3	**Scale.** "Bookings are rows in a database, and we run several servers."
4	**Algorithm.** "Given a day's meetings, what's the fewest rooms needed so all can happen?"
5	**Defend it.** "Why a TreeMap per room? A list of meetings is simpler."

Problem 4 — Chess

Heavy · 100 minutes (25 design + 75 code) · machine-coding format

> **The interviewer says:** "Design a chess game for two players. Every piece moves by its own rules, players alternate, pieces get captured, and you can never leave your own king in check. Detect check and checkmate."

After your questions, the requirements are

R1	The game starts from the standard position; White moves first. Squares are a1 to h8 (file letter, rank number).
R2	Each piece moves by its own rule (table below). Rooks, bishops and queens can't jump over pieces; knights can. No piece can land on a piece of its own colour.
R3	A move is refused, checked in this order, if: a square is badly written, the game is over, there's no piece on the first square, the piece isn't the mover's colour, the move breaks the piece's rule, or it would leave the mover's own king in check.
R4	Moving onto an enemy piece captures it. A pawn reaching the last rank becomes a queen.
R5	After each move, look at the other side. If it has no legal move: **checkmate** if its king is in check, otherwise **stalemate** (a draw); either way the game is over. If it has moves but its king is attacked, announce **check**.
R6	For testing, EMPTY clears the board (White to move) and PLACE puts a piece on a square: upper case for White (K, Q, R, B, N, P), lower case for Black.

Piece	Moves
King	one square in any direction
Queen	any distance in a straight line or diagonal, nothing in between
Rook / Bishop	any distance along a rank or file / along a diagonal, nothing in between
Knight	an L: two squares one way and one square sideways; may jump
Pawn	one square forward to an empty square; two forward from its starting rank if both squares are empty; captures one square diagonally forward

NF1	A new kind of piece (for a chess variant) should be one new class.
NF2	The rules must not depend on how the game is shown (console today, an app tomorrow).

Out of scope: castling, en passant, choosing the promotion piece, draws by repetition or the 50-move rule, clocks (Part C asks about some).

Commands and exact output

Command	Output
`NEW`	`New game, White to move`
`EMPTY`	`Empty board, White to move`
`PLACE square letter`	`Placed white king on e1`
`MOVE from to`	`White pawn e2-e4` · `White queen h5xf7, takes pawn` · `White pawn a7-a8, promotes to queen`, then if needed one of: `Black is in check` · `Checkmate! White wins` · `Stalemate! It's a draw`
`BOARD`	rank 8 first: `8 r . b q k b . r` … `1 R N B . K . N R`, then `a b c d e f g h` (two spaces first; `.` is empty)

Error	Output
`bad square / unknown piece letter`	`Invalid square z9` · `Unknown piece X`
`game over / empty square / wrong colour`	`Game is over` · `No piece at e3` · `It's White's turn`
`against the piece's rule`	`Illegal move for knight: g1 -> g3`
`would leave your king attacked`	`That move leaves your king in check`

Part A — design

1	Classes and who owns what: game, board, square, piece, the six kinds, colour, a move's result.
2	How does each piece know its moves without a big `switch` in the game? What can the pieces share?
3	"This move would leave my king in check": how do you find out, with the code you already have?
4	Checkmate and stalemate: what one question answers both?

Part B — code

input.txt

```
NEW
MOVE e2 e5
MOVE e7 e5
MOVE e3 e4
MOVE z9 e4
MOVE g1 g3
MOVE e2 e4
MOVE e7 e5
MOVE f1 c4
MOVE b8 c6
MOVE d1 h5
MOVE g8 f6
MOVE h5 f7
MOVE e8 f7
BOARD
EMPTY
PLACE e1 K
PLACE e2 N
PLACE e7 r
PLACE a8 k
PLACE c3 X
MOVE e2 c3
MOVE e1 d1
MOVE e7 d7
MOVE d1 d2
MOVE d1 c1
EMPTY
PLACE a7 P
PLACE h1 K
PLACE e8 k
MOVE a7 a8
EMPTY
PLACE h8 k
PLACE f7 K
PLACE g5 Q
MOVE g5 g6
```

expected output

```
New game, White to move
Illegal move for pawn: e2 -> e5
It's White's turn
No piece at e3
Invalid square z9
Illegal move for knight: g1 -> g3
White pawn e2-e4
Black pawn e7-e5
White bishop f1-c4
Black knight b8-c6
White queen d1-h5
Black knight g8-f6
White queen h5xf7, takes pawn
Checkmate! White wins
Game is over
8 r . b q k b . r
7 p p p p . Q p p
6 . . n . . n . .
5 . . . . p . . .
4 . . B . P . . .
3 . . . . . . . .
2 P P P P . P P P
1 R N B . K . N R
  a b c d e f g h
Empty board, White to move
Placed white king on e1
Placed white knight on e2
Placed black rook on e7
Placed black king on a8
Unknown piece X
That move leaves your king in check
White king e1-d1
Black rook e7-d7
White is in check
That move leaves your king in check
White king d1-c1
Empty board, White to move
Placed white pawn on a7
Placed white king on h1
Placed black king on e8
White pawn a7-a8, promotes to queen
Black is in check
Empty board, White to move
Placed black king on h8
Placed white king on f7
Placed white queen on g5
White queen g5-g6
Stalemate! It's a draw
```

What the test covers. The first game is Scholar's Mate: checkmate in four moves. The second position pins a knight to its king (moving it is refused), then gives check. The third promotes a pawn, which gives check. The fourth is a stalemate: Black's king can't move, but isn't in check.

Part C — pressure questions

1	**Change.** "Add undo: take back the last move, as many times as you like."
2	**Change.** "Add castling." What does the game need to remember that it doesn't today?
3	**Performance.** "Checking for mate tries every move on the board. Is that too slow?"
4	**Scale.** "Make it online: two players on phones, thousands of games."
5	**Defend it.** "One class per piece is overkill. A switch on the piece type in one method does the same."

Hints

Read one level at a time, and only after 15 minutes of being stuck. Each level gives away more than the last.

Problem 1 — Snake and Ladder

Hint 1 (a nudge). A snake and a ladder do the same thing: "if you land on cell X, go to cell Y". They differ only in direction.

Hint 2 (the shape). `Board` (size, a map from start cell to end cell) · `Player` (name, position) · `Game` (players, whose turn, started, over, `roll`). Snakes and ladders are entries in one map, checked when added.

Hint 3 (the key decision). Loop check when adding X → Y: start at Y and follow the map; if you reach X, refuse. In `roll`: overshoot first, then move, then a `while` loop over the map, then win, then "a six rolls again", and only then the next player's turn.

Problem 2 — Cache with TTL

Hint 1. Never read the system clock inside the cache: take a `Clock` in the constructor. Expired entries are found lazily on reads and actively by a sweep.

Hint 2. A `LinkedHashMap` in access order gives the least recently used (its first entry). A `TreeSet` of entries sorted by expiry time (then a sequence number, to break ties) gives the soonest to expire. Every add and remove touches both.

Hint 3. One private helper, `live(key)`: get the entry; if it has expired, remove it, count it, and treat it as missing. GET and TTL both use it. For room in `put`: look at the first entry of the expiry set; if it has expired, remove it, otherwise remove the map's eldest entry.

Problem 3 — Meeting Room Scheduler

Hint 1. Each room owns its own calendar, and its own lock. Two slots [s1, e1) and [s2, e2) clash exactly when s1 < e2 and s2 < e1.

Hint 2. `Room` (name, seats, a `TreeMap` from start minute to meeting) · a `Meeting` record · `Scheduler` (rooms, meetings by ID, an `AtomicInteger` for IDs, a `Comparator` for "which room first"). A small `Time` helper converts HH:MM to minutes and back.

Hint 3. Meetings in one room never overlap, so only the last meeting starting before your end can reach into your slot: `byStart.lowerEntry(end)`, then check whether its end is after your start. `book()` on the room is `synchronized`: check, then add. BOOK_ANY tries the rooms in order and moves on when one says "busy".

Problem 4 — Chess

Hint 1. Each piece answers one question: "can I go from here to there on this board?" Check and checkmate are the game's job, built on top of that question.

Hint 2. An abstract `Piece` with `canMove(board, from, to)` and six subclasses; a shared `pathClear` helper for rook, bishop and queen. A `Square` record, a `Color` enum, a `Board` with an 8 × 8 array and `isAttacked(square, byColour)`. `Game` holds the turn and runs the checks.

Hint 3. "Leaves my king in check": make the move on the board, ask whether your king's square is attacked, then put both squares back. "Has any legal move": try every piece of that colour to every square, using the same two checks. No legal move and in check: mate; no legal move, not in check: stalemate.

Solution 1 — Snake and Ladder

Questions a strong candidate asks first

Question	Answer assumed here
Board size? How many players and dice?	Any size; 2 or more players; one dice
What if a roll goes past the last cell?	The player stays
Can a snake end on a ladder (a chain)?	Yes, but loops are refused
Does a six give another turn?	Yes (three sixes: out of scope)
Where do dice values come from?	The input, so tests repeat exactly

Part A — model answer

Entity	Kind	Owns
`Board`	class	size and one map, start cell → end cell, for snakes and ladders; checks them as they're added
`Player`	class	name and position
`Game`	class	players in order, whose turn, started or over; `roll`

```
Game 1 ———> 1 Board (jumps: Map<Integer, Integer>)        Game 1 ◆——— 2..* Player
```

1. One map, not two classes. A snake and a ladder are the same rule, "land on X, go to Y", in different directions. Two classes would hold identical code. Cells don't need objects either: a cell is just a number. The direction matters only when adding (to check the input) and when printing (bigger is a ladder).

2. Who owns what. "Where does 27 send me?" belongs to the **board**: it holds the map. "Whose turn is it?" belongs to the **game**: it holds the players and the turn. The player only knows its own position.

3. The loop check. Adding X → Y: start at Y and follow the map. Reaching a plain cell means it's safe; reaching X means landing on X would never end. It's the same idea as refusing cycles in a Composite tree.

4. NF1: a Dice interface. `interface Dice { int roll(); }`, with `RandomDice` for real games, a fixed sequence for tests, `TwoDice` later. The game asks the dice it was given. In this test the value comes in the command, which is the same idea: the game never decides randomness itself.

In the interview: write this first. (1) `Board` with the map and `jumpFrom`, skipping the checks. (2) `Game.roll`: overshoot, move, follow jumps, win, turn: about 15 minutes. (3) The driver. (4) The board's checks, the loop check last. The domain code here is about 95 lines.

Part B — the code

```
import java.util.*;

class Board {
    final int size;
    private final Map<Integer, Integer> jumps = new HashMap<>();     // start -> end, snakes and ladders

    Board(int size) { this.size = size; }

    void addSnake(int head, int tail)    { add(head, tail, "snake"); }
    void addLadder(int bottom, int top)  { add(bottom, top, "ladder"); }

    private void add(int from, int to, String kind) {
        for (int cell : new int[]{from, to})
            if (cell < 1 || cell > size)
                throw new IllegalArgumentException("Cell " + cell + " is off the board");
        if (kind.equals("snake") && to >= from)
            throw new IllegalArgumentException("A snake must go down");
        if (kind.equals("ladder") && to <= from)
            throw new IllegalArgumentException("A ladder must go up");
        if (from == size) throw new IllegalArgumentException("Nothing can start at " + size);
        if (jumps.containsKey(from))
            throw new IllegalArgumentException("Cell " + from + " already has a snake or ladder");
        for (int c = to; c != from; c = jumps.get(c))      // follow the chain from the end
            if (!jumps.containsKey(c)) {
                jumps.put(from, to);
                return;
            }
        throw new IllegalArgumentException("That would make a loop");
    }

    Integer jumpFrom(int cell) { return jumps.get(cell); }   // null: no snake or ladder here
}

class Player {
    final String name;
    int position = 0;                                                  // 0 = not on the board yet

    Player(String name) { this.name = name; }
}

class Game {
    private final Board board;
    private final List<Player> players = new ArrayList<>();
    private int turn = 0;
```

```
    private boolean started, over;

    Game(Board board) { this.board = board; }

    void join(String name) {
        if (started) throw new IllegalStateException("Game already started");
        if (players.stream().anyMatch(p -> p.name.equals(name)))
            throw new IllegalArgumentException("Player " + name + " already exists");
        players.add(new Player(name));
    }

    List<Player> start() {
        if (players.size() < 2) throw new IllegalStateException("Need at least 2 players");
        started = true;
        return players;
    }

    String roll(int dice) {                              // returns what happened, as one line
        if (!started) throw new IllegalStateException("Game not started");
        if (over) throw new IllegalStateException("Game is over");
        if (dice < 1 || dice > 6) throw new IllegalArgumentException("Roll must be 1 to 6");
        Player p = players.get(turn);
        StringBuilder say = new StringBuilder(p.name + " rolled " + dice + ": ");
        int target = p.position + dice;
        if (target > board.size) {
            say.append("stays at ").append(p.position)
               .append(" (needs exactly ").append(board.size - p.position).append(")");
        } else {
            say.append(p.position).append(" -> ").append(target);
            Integer next;
            while ((next = board.jumpFrom(target)) != null) {          // jumps can chain
                say.append(next > target ? ", ladder to " : ", snake to ").append(next);
                target = next;
            }
            p.position = target;
        }
        if (p.position == board.size) {
            over = true;
            return say.append(". ").append(p.name).append(" wins!").toString();
        }
        if (dice == 6) return say.append(", rolls again").toString();  // a six gives another turn
        turn = (turn + 1) % players.size();
        return say.toString();
    }

    String positions() {
        StringJoiner j = new StringJoiner(", ");
        for (Player p : players) j.add(p.name + " " + p.position);
        return j.toString();
    }
}

public class Main {
    public static void main(String[] args) {
        Board board = null;
        Game game = null;
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
            try {
                switch (p[0]) {
                    case "BOARD" -> {
                        board = new Board(Integer.parseInt(p[1]));
                        game = new Game(board);
                        System.out.println("Board of " + p[1] + " cells");
                    }
                    case "SNAKE" -> {
                        board.addSnake(Integer.parseInt(p[1]), Integer.parseInt(p[2]));
                        System.out.println("Snake " + p[1] + " -> " + p[2]);
                    }
                    case "LADDER" -> {
                        board.addLadder(Integer.parseInt(p[1]), Integer.parseInt(p[2]));
                        System.out.println("Ladder " + p[1] + " -> " + p[2]);
                    }
                    case "PLAYER" -> {
                        game.join(p[1]);
                        System.out.println("Player " + p[1] + " joined");
                    }
                    case "START" -> {
                        StringJoiner names = new StringJoiner(", ");
                        for (Player pl : game.start()) names.add(pl.name);
                        System.out.println("Game started: " + names);
                    }
                    case "ROLL" -> System.out.println(game.roll(Integer.parseInt(p[1])));
                    case "POSITIONS" -> System.out.println(game.positions());
                    default -> System.out.println("Unknown command: " + p[0]);
                }
            } catch (IllegalArgumentException | IllegalStateException e) {
                System.out.println(e.getMessage());
            }
        }
    }
}
```

How to read it: `roll` follows R4–R6 in order and builds its line as it goes. The order of the last three steps matters: check for a win first, then "a six rolls again", and only then pass the turn. The loop check in `add` follows the chain from the new end before storing anything.

Part C — model answers

1. Three sixes. The game remembers two more things: the position where this turn began, and how many sixes in a row. On the third six, put the player back at that position, reset the count, and pass the turn. Only `Game` changes; the board doesn't.

2. Special cells. Now cells do different things, so the map's value becomes an object: `interface CellEffect { void apply(Player p, Game g); }`, with `Jump(to)`, `SkipNextTurn` and `RollAgain`. The board returns the effect for a cell, and the game applies it. Snakes and ladders become `Jump`s. This is the point where behaviour, not just data, differs, so classes earn their place.

3. Testing with a random dice. Pass the dice in. Tests use a fixed sequence (`new FixedDice(3, 6, 5)`) or a seeded random; the real game uses an unseeded one. This was run:

```
interface Dice { int roll(); }

class RandomDice implements Dice {
    private final Random random;
    RandomDice(Random random) { this.random = random; }
    public int roll() { return random.nextInt(6) + 1; }
}
```

4. Online. The server owns the game and rolls the dice, so a phone can't cheat. Each game is independent: give it its own lock, or a single thread that handles its commands in order. Keep game state in a store such as Redis so any server can pick it up, and push each move to both phones over WebSockets.

5. "Why not Snake and Ladder classes?" "You'd be right if they behaved differently; then they'd be classes. Today they're the same rule in opposite directions, so two classes would duplicate the same line. When special cells arrive, I'd introduce a `CellEffect` interface, and snakes and ladders would both become one `Jump` class."

Common mistakes

- Following only one jump, so a ladder that ends on another ladder stops halfway.
- Allowing a loop (a snake back to a ladder's bottom that leads to the snake), so the game never ends.
- Passing the turn after a six, or after the winning move.
- Moving past the last cell instead of staying.
- Creating `new Random()` inside the game, so it can't be tested.

Score yourself (1 point each)

☐	Snakes and ladders live in one map	☐	Additions are checked in the stated order
☐	Loops are refused	☐	Jumps are followed until a plain cell
☐	An overshoot leaves the player in place	☐	A six gives another turn, but not after a win
☐	The game ends at exactly the last cell	☐	Your output matches exactly
☐	You can say how the dice becomes replaceable	☐	Part C 2: you know when cells become objects

Solution 2 — Cache with TTL

Questions a strong candidate asks first

Question	Answer assumed here
Is the TTL per entry or one for the whole cache?	Per entry; 0 means never
When it's full, what goes first?	An expired entry if there is one, else the least recently used
Does a put on an existing key reset its TTL?	Yes
How do we test expiry without waiting?	Pass a clock in
How precise is time? Many threads?	Whole seconds; yes, many threads

Part A — model answer

```
TtlCache<K, V> ———> «interface» Clock  ◁- - -  ManualClock (tests), SystemClock (production)
TtlCache<K, V> ———> LinkedHashMap<K, Entry>   access order: first entry = least recently used
TtlCache<K, V> ———> TreeSet<Entry>           by expiry time, then sequence: first = soonest to expire
Entry: key, value, expiresAt, seq
```

1. Lazy and active expiry. **Lazy**: check the time when someone reads the key. It's cheap and guarantees an expired value is never returned, but an entry nobody reads stays in memory forever. **Active**: a regular sweep removes what has expired. It frees memory, at the cost of work on a timer. Real caches, Redis among them, do both.

2. Two orders, two structures. The access-order `LinkedHashMap` keeps "least recently used" at the front. The `TreeSet` keeps "soonest to expire" at the front, and the sweep simply removes from the front while the first entry has expired. Every add and remove must touch both, so one private `remove(entry)` does it in one place.

3. The clock is passed in. Tests pass a `ManualClock` they can move forward; production passes a clock that reads system time. The cache never knows which. It's dependency injection, and it's what makes expiry testable in milliseconds instead of minutes.

4. One lock; the sweep on its own thread. Every public method is `synchronized`, because even a read changes the order and may remove an entry. In production a `ScheduledExecutorService` calls `cleanup()` every second on a background thread (Part C 1).

In the interview: write this first. (1) `Clock` and the `Entry`. (2) `put` and `get` with lazy expiry (the `live` helper): 15 minutes. (3) Eviction when full. (4) `cleanup`. (5) TTL and stats. The domain code here is about 120 lines.

Part B — the code

```
import java.util.*;

interface Clock {                               // injected, so tests control time
    long now();
}

class ManualClock implements Clock {
    private long time;

    public synchronized long now() { return time; }

    synchronized void set(long t) {
        if (t < time) throw new IllegalArgumentException("Time cannot go back");
        time = t;
    }
}

class TtlCache<K, V> {
    private static final long NEVER = Long.MAX_VALUE;

    private class Entry {
        final K key;
        final long seq;                         // breaks ties between equal expiry times
        V value;
        long expiresAt;

        Entry(K key, V value, long expiresAt, long seq) {
            this.key = key; this.value = value; this.expiresAt = expiresAt; this.seq = seq;
        }
    }

    private final int capacity;
    private final Clock clock;
    // Access order: the eldest entry is the least recently used.
    private final LinkedHashMap<K, Entry> data = new LinkedHashMap<>(16, 0.75f, true);
    private final TreeSet<Entry> byExpiry = new TreeSet<>(
            Comparator.comparingLong((Entry e) -> e.expiresAt).thenComparingLong(e -> e.seq));
    private long seq;
    private int hits, misses, expired, evictions;

    TtlCache(int capacity, Clock clock) {
        if (capacity < 1) throw new IllegalArgumentException("Capacity must be at least 1");
        this.capacity = capacity;
        this.clock = clock;
    }
```

```
    synchronized Optional<V> get(K key) {
        Entry e = live(key);
        if (e == null) { misses++; return Optional.empty(); }
        hits++;
        return Optional.of(e.value);
    }

    // Returns the seconds left (NEVER = no expiry), or empty for a miss.
    synchronized Optional<Long> ttl(K key) {
        Entry e = live(key);
        if (e == null) return Optional.empty();
        return Optional.of(e.expiresAt == NEVER ? NEVER : e.expiresAt - clock.now());
    }

    // Returns a note about the entry removed to make room, or "" if none.
    synchronized String put(K key, V value, long ttlSeconds) {
        if (ttlSeconds < 0) throw new IllegalArgumentException("TTL cannot be negative");
        long expiresAt = ttlSeconds == 0 ? NEVER : clock.now() + ttlSeconds;
        String note = "";
        Entry old = data.get(key);
        if (old != null) {
            remove(old);
        } else if (data.size() == capacity) {
            Entry soonest = byExpiry.first();
            if (isExpired(soonest)) {             // an expired entry goes first...
                remove(soonest);
                expired++;
                note = ", evicted " + soonest.key + " (expired)";
            } else {                              // ...otherwise the least recently used
                Entry lru = data.values().iterator().next();
                remove(lru);
                evictions++;
                note = ", evicted " + lru.key + " (least recent)";
            }
        }
        Entry e = new Entry(key, value, expiresAt, seq++);
        data.put(key, e);
        byExpiry.add(e);
        return note;
    }

    // What a background job would run every few seconds: remove everything that has expired.
    synchronized List<K> cleanup() {
        List<K> removed = new ArrayList<>();
        while (!byExpiry.isEmpty() && isExpired(byExpiry.first())) {
            Entry e = byExpiry.first();
            remove(e);
            expired++;
            removed.add(e.key);
        }
        return removed;
    }

    synchronized String stats() {
        return "Size " + data.size() + " of " + capacity + ", hits " + hits + ", misses " + misses
                + ", expired " + expired + ", evictions " + evictions;
    }

    // A read counts as a use; an expired entry is removed and treated as missing.
    private Entry live(K key) {
        Entry e = data.get(key);
        if (e != null && isExpired(e)) {
            remove(e);
            expired++;
            return null;
        }
        return e;
    }

    private boolean isExpired(Entry e) { return e.expiresAt <= clock.now(); }

    private void remove(Entry e) {
        data.remove(e.key);
        byExpiry.remove(e);
    }
}

public class Main {
    public static void main(String[] args) {
        ManualClock clock = new ManualClock();
        TtlCache<String, String> cache = null;
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
            try {
                switch (p[0]) {
                    case "CREATE" -> {
                        cache = new TtlCache<>(Integer.parseInt(p[1]), clock);
                        System.out.println("Cache ready, capacity " + p[1]);
                    }
                    case "TIME" -> {
                        clock.set(Long.parseLong(p[1]));
                        System.out.println("Time " + p[1]);
                    }
                    case "PUT" -> {
                        long ttl = Long.parseLong(p[3]);
                        String note = cache.put(p[1], p[2], ttl);
                        String expiry = ttl == 0 ? "no expiry"
                                                 : "expires at " + (clock.now() + ttl);
```

```
                    System.out.println("Put " + p[1] + ", " + expiry + note);
                }
                case "GET" -> System.out.println(
                        cache.get(p[1]).map(v -> p[1] + " = " + v).orElse(p[1] + ": miss"));
                case "TTL" -> System.out.println(cache.ttl(p[1])
                        .map(s -> p[1] + ": "
                                + (s == Long.MAX_VALUE ? "no expiry" : s + "s left"))
                        .orElse(p[1] + ": miss"));
                case "CLEANUP" -> {
                    List<String> removed = cache.cleanup();
                    System.out.println(removed.isEmpty() ? "Cleanup removed nothing"
                            : "Cleanup removed " + removed.size() + ": "
                                    + String.join(", ", removed));
                }
                case "STATS" -> System.out.println(cache.stats());
                default -> System.out.println("Unknown command: " + p[0]);
            }
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
    }
  }
}
```

How to read it: `live(key)` is the only way reads reach an entry, so lazy expiry can't be forgotten. The `seq` number gives two entries with the same expiry time a fixed order, so the `TreeSet` never treats them as the same entry. "Never expires" is stored as `Long.MAX_VALUE`, so those entries sort last.

Part C — model answers

1. Cleanup every second. A single background thread, started with the cache and stopped on shutdown. This was run against the cache above:

```
ScheduledExecutorService sweeper = Executors.newSingleThreadScheduledExecutor();
sweeper.scheduleAtFixedRate(() -> {
    List<String> gone = cache.cleanup();
    if (!gone.isEmpty()) System.out.println("swept " + gone);
}, 1, 1, TimeUnit.SECONDS);
```

2. A long sweep blocks readers. Sweep in small batches: remove at most, say, 100 entries per lock, release, and repeat. Redis takes this approach, checking a small random sample of keys that have a TTL several times a second instead of scanning them all. Or split the cache into stripes by key hash, each with its own lock and its own sweep. In production, a library such as Caffeine handles expiry with a timer wheel, without scanning.

3. Sliding expiry. Store each entry's TTL as well as its expiry time. On a read of a sliding key, compute a new expiry (now + TTL), and move the entry in the expiry set: remove it, change the time, add it back. Never change the time while the entry is inside the `TreeSet`, or the set loses track of it.

4. 500 misses at once (a stampede). Let only one request load the value while the others wait for it: keep a map of key → future, and use `computeIfAbsent` so exactly one loader runs. Two more habits help: add a little random jitter to TTLs, so popular keys don't all expire at the same second, and serve the old value briefly while one request refreshes it.

5. "Why a Clock interface?" "You'd be right that production could read the system clock directly. But then every expiry test has to sleep, which makes the suite slow and flaky. With a clock passed in, a test moves time to 26 in one line. It also gives one place to handle the system clock jumping backwards after a time sync; for measuring durations, a monotonic clock such as `System.nanoTime()` is safer."

Common mistakes

- Returning an expired value because only the sweep removes entries.
- Calling `System.currentTimeMillis()` inside the cache, so tests must sleep.
- Changing an entry's expiry while it's inside the `TreeSet`.
- Removing from one structure but not the other.
- A `TreeSet` compared only by expiry time, so two keys expiring together look like duplicates and one is lost.
- Evicting the least recently used while an expired entry is still taking up room.

Score yourself (1 point each)

☐	Time comes from a clock passed in	☐	An expired entry is never returned
☐	Reads use one helper that expires lazily	☐	Two structures: last use and expiry order
☐	Ties in expiry time can't lose an entry	☐	Full: expired first, then least recently used
☐	Cleanup removes in expiry order	☐	Your output matches exactly
☐	Part C 1: you can schedule the sweep	☐	Part C 4: you named a fix for a stampede

Solution 3 — Meeting Room Scheduler

Questions a strong candidate asks first

Question	Answer assumed here
One day, or a calendar of many days?	One day; times HH:MM
Does a meeting ending at 11:00 clash with one starting at 11:00?	No: slots are [start, end)
For "any room", which room is best?	The smallest that fits; then by name
Recurring meetings? Invitations?	Not now (Part C)
Many people booking at once?	Yes

Part A — model answer

```
Scheduler ———> * Room 1 ◆——— * Meeting       Room: name, seats, TreeMap<Integer, Meeting> (by start)
Scheduler ———> Map<String, Meeting> (by ID)   Scheduler ———> Comparator<Room> (which room first)
Meeting: record (id, room, start, end, organiser, people)       Time: HH:MM ⇄ minutes
```

1. Each room owns its calendar. A `TreeMap` from start minute to meeting keeps the room's meetings sorted, so the schedule comes out in order for free, and a clash check is one lookup.

2. The clash condition. [s1, e1) and [s2, e2) clash exactly when **s1 < e2 and s2 < e1**. Because one room's meetings never overlap each other, only the last meeting starting before your end can reach into your slot: `lowerEntry(end)`, then check whether its end is after your start. O(log n).

3. One step per room. "Check for a clash" + "add the meeting" happen inside the room's `synchronized book()`. The lock is per room, so bookings in different rooms never wait. BOOK_ANY needs no extra lock: it tries the rooms in order, and each try is already one safe step; if a room turns out busy, it moves to the next. Tested: with the lock removed, RUSH showed "2 booked" in **13 runs out of 100**.

4. NF2: the preference is a Comparator. "Smallest that fits, then by name" is one `Comparator<Room>` in the scheduler. "Nearest floor first" or "has a projector" is a different comparator (or a filter plus a comparator): a small Strategy, with no change to the booking code.

In the interview: write this first. (1) `Room` with the `TreeMap`, the clash check and `synchronized book`: 20 minutes. (2) The `Scheduler`'s book and cancel. (3) The driver and the time helper. (4) BOOK_ANY and FREE, which reuse the same pieces. (5) RUSH, with the given helper. The domain code here is about 115 lines.

Part B — the code

```
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.IntPredicate;

record Meeting(String id, String room, int start, int end, String organiser, int people) {
    String slot() { return Time.format(start) + "-" + Time.format(end); }
}

final class Time {                                      // minutes since midnight <-> "HH:MM"
    private Time() {}

    static int parse(String hhmm) {
        if (!hhmm.matches("\\d{2}:\\d{2}"))
            throw new IllegalArgumentException("Invalid time " + hhmm);
        int h = Integer.parseInt(hhmm.substring(0, 2)), m = Integer.parseInt(hhmm.substring(3));
        if (h > 23 || m > 59) throw new IllegalArgumentException("Invalid time " + hhmm);
        return h * 60 + m;
    }

    static String format(int minutes) {
        return String.format("%02d:%02d", minutes / 60, minutes % 60);
    }
}

class Room {                                            // its calendar is guarded by its own lock
    final String name;
    final int capacity;
    private final TreeMap<Integer, Meeting> byStart = new TreeMap<>();

    Room(String name, int capacity) { this.name = name; this.capacity = capacity; }

    // The meeting that clashes with [start, end), if any. Meetings never overlap, so only the
    // last one starting before `end` can reach past `start`.
    private Meeting clash(int start, int end) {
        Map.Entry<Integer, Meeting> before = byStart.lowerEntry(end);
        return (before != null && before.getValue().end() > start) ? before.getValue() : null;
    }

    synchronized boolean isFree(int start, int end) { return clash(start, end) == null; }

    synchronized Meeting book(int start, int end, String organiser, int people, AtomicInteger ids) {
        Meeting busy = clash(start, end);               // check and add: one step under the lock
        if (busy != null)
            throw new IllegalStateException(name + " is busy " + busy.slot()
                    + " (" + busy.id() + ")");
        Meeting m = new Meeting("M" + ids.incrementAndGet(), name, start, end, organiser, people);
```

```
        byStart.put(start, m);
        return m;
    }

    synchronized void cancel(Meeting m)        { byStart.remove(m.start()); }
    synchronized List<Meeting> schedule()      { return new ArrayList<>(byStart.values()); }
}

class Scheduler {
    private final Map<String, Room> rooms = new ConcurrentHashMap<>();
    private final Map<String, Meeting> meetings = new ConcurrentHashMap<>();
    private final AtomicInteger ids = new AtomicInteger();
    // Which room to try first for "any room": the smallest that fits, then by name.
    private final Comparator<Room> preference =
            Comparator.comparingInt((Room r) -> r.capacity).thenComparing(r -> r.name);

    void addRoom(String name, int capacity) {
        if (rooms.putIfAbsent(name, new Room(name, capacity)) != null)
            throw new IllegalArgumentException("Room " + name + " already exists");
    }

    Meeting book(String roomName, int start, int end, String organiser, int people) {
        Room room = room(roomName);
        checkSlot(start, end);
        if (people > room.capacity)
            throw new IllegalArgumentException(room.name + " seats only " + room.capacity);
        return remember(room.book(start, end, organiser, people, ids));
    }

    Meeting bookAny(int start, int end, String organiser, int people) {
        checkSlot(start, end);
        for (Room room : candidates(people)) {
            try {
                return remember(room.book(start, end, organiser, people, ids));
            } catch (IllegalStateException busy) {
                // taken: try the next room
            }
        }
        throw new IllegalStateException("No room free for " + people + " people at "
                + Time.format(start) + "-" + Time.format(end));
    }

    List<Room> freeRooms(int start, int end, int people) {
        checkSlot(start, end);
        return candidates(people).stream().filter(r -> r.isFree(start, end)).toList();
    }

    Meeting cancel(String id) {
        Meeting m = meetings.remove(id);
        if (m == null) throw new IllegalArgumentException("Unknown meeting " + id);
        rooms.get(m.room()).cancel(m);
        return m;
    }

    Room room(String name) {
        Room r = rooms.get(name);
        if (r == null) throw new IllegalArgumentException("Unknown room " + name);
        return r;
    }

    private List<Room> candidates(int people) {
        return rooms.values().stream()
                .filter(r -> r.capacity >= people).sorted(preference).toList();
    }

    private Meeting remember(Meeting m) {
        meetings.put(m.id(), m);
        return m;
    }

    private void checkSlot(int start, int end) {
        if (end <= start) throw new IllegalArgumentException("End must be after start");
    }
}

public class Main {
    public static void main(String[] args) throws Exception {
        Scheduler scheduler = new Scheduler();
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
            try {
                switch (p[0]) {
                    case "ROOM" -> {
                        scheduler.addRoom(p[1], Integer.parseInt(p[2]));
                        System.out.println("Room " + p[1] + " added, seats " + p[2]);
                    }
                    case "BOOK" -> print(scheduler.book(p[1], Time.parse(p[2]), Time.parse(p[3]),
                            p[4], Integer.parseInt(p[5])));
                    case "BOOK_ANY" -> print(scheduler.bookAny(Time.parse(p[1]), Time.parse(p[2]),
                            p[3], Integer.parseInt(p[4])));
                    case "FREE" -> {
                        List<Room> free = scheduler.freeRooms(Time.parse(p[1]), Time.parse(p[2]),
                                Integer.parseInt(p[3]));
                        StringJoiner names = new StringJoiner(", ");
                        for (Room r : free) names.add(r.name + " (" + r.capacity + ")");
                        System.out.println(free.isEmpty() ? "No room free" : "Free: " + names);
                    }
                    case "CANCEL" -> System.out.println("Cancelled " + scheduler.cancel(p[1]).id());
```

```
                    case "SCHEDULE" -> {
                        List<Meeting> day = scheduler.room(p[1]).schedule();
                        if (day.isEmpty()) System.out.println(p[1] + " is free all day");
                        for (Meeting m : day)
                            System.out.println(m.slot() + " " + m.id() + " " + m.organiser()
                                    + " (" + m.people() + ")");
                    }
                    case "RUSH" -> {                    // n people, same room, same slot, at once
                        int start = Time.parse(p[2]), end = Time.parse(p[3]);
                        int n = Integer.parseInt(p[4]);
                        int booked = rush(n, i -> {
                            try {
                                scheduler.book(p[1], start, end, "team", 1);
                                return true;
                            } catch (IllegalStateException busy) {
                                return false;
                            }
                        });
                        System.out.println("Rush: " + booked + " booked, " + (n - booked)
                                + " refused");
                    }
                    default -> System.out.println("Unknown command: " + p[0]);
                }
            } catch (IllegalArgumentException | IllegalStateException e) {
                System.out.println(e.getMessage());
            }
        }
    }

    static void print(Meeting m) {
        System.out.println(m.id() + ": " + m.room() + " " + m.slot() + ", " + m.organiser() + ", "
                + m.people() + " people");
    }

    // Given: runs task(0) .. task(n-1) on n threads that all start together; counts the trues.
    static int rush(int n, IntPredicate task) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(n);
        CountDownLatch go = new CountDownLatch(1);
        List<Future<Boolean>> results = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            int id = i;
            results.add(pool.submit(() -> { go.await(); return task.test(id); }));
        }
        go.countDown();
        int trues = 0;
        for (Future<Boolean> f : results) if (f.get()) trues++;
        pool.shutdown();
        return trues;
    }
}
```

How to read it: `Room.clash` is the whole interval logic; `bookAny` catches only "busy" and lets every other error through.

Part C — model answers

1. Weekly recurring meetings. Add a date to every slot, and a `Series` (rule, start date, count) that produces the dates. Book all-or-nothing under the room's lock: check every date, then add them all. Each meeting keeps its series ID, so you can cancel one or all.

2. Invitations and emails. Observer: a `MeetingListener` with `onBooked` and `onCancelled`, and an email notifier. Call listeners after the booking succeeds and outside the room's lock; a failed email must not undo the booking, so retry it from a queue.

3. A database and several servers. `synchronized` protects one JVM. In one transaction: lock the room's row (`SELECT … FOR UPDATE`), check for a clash with `WHERE room = ? AND start_time < :end AND end_time > :start`, then insert. PostgreSQL can also enforce it for you with an exclusion constraint on the time range, so two overlapping inserts can't both succeed.

4. Fewest rooms for a day. Sort the meetings by start time. Keep a min-heap of the end times of meetings in progress. For each meeting, remove every end time that's at or before its start, then add its end. The largest the heap ever gets is the answer. O(n log n).

5. "A list is simpler." "You'd be right for a few meetings a day: scanning a list is microseconds. The `TreeMap` costs no extra code here: the clash check is one lookup and the schedule is already sorted. And when the calendar grows to months of bookings per room, the list's scan grows with it, while the lookup stays O(log n)."

Common mistakes
- Clash checks with ≤ instead of <, so back-to-back meetings are refused.
- Checking only whether the new start falls inside a meeting, missing a new meeting that covers an existing one.
- Checking and adding in two separately locked steps.
- Choosing a room for BOOK_ANY, then booking it later in a separate step without handling "it just got taken".

Score yourself (1 point each)

☐	Each room owns a sorted calendar	☐	The clash rule treats slots as [start, end)
☐	The clash check is one lookup	☐	Check and add in one locked step, per room
☐	BOOK_ANY: smallest that fits, then by name	☐	BOOK_ANY moves on when a room is busy
☐	Errors in the stated order	☐	Your output matches exactly, RUSH included
☐	Part C 3: you described a safe database booking	☐	Part C 4: you solved the fewest-rooms question

Solution 4 — Chess

Questions a strong candidate asks first

Question	Answer assumed here
Full rules, or a core set?	All pieces, check, mate, stalemate, promotion to a queen
Castling? En passant? Draw rules?	Out of scope for now (Part C 2)
Two humans, or a computer player?	Two humans
How are moves entered?	Two squares: `e2 e4`
Do we need test positions?	Yes: an empty board and piece placement

Part A — model answer

```
Game 1 ———> 1 Board 1 ◆——— 0..32 Piece                Game: turn (Color), over
«abstract» Piece (color, canMove)  ◁———  King, Queen, Rook, Bishop, Knight, Pawn
Board: Piece[8][8], at, put, squaresOf(color), isAttacked(square, byColor)
Square: record (file, rank)    Color: enum    MoveResult: record    Outcome: enum
```

1–2. Each piece knows its own moves. `canMove(board, from, to)` is abstract, and each subclass answers by its own rule. The game never switches on the piece type; that's polymorphism, and NF1 is one new class. The pieces share `pathClear` (for rook, bishop and queen) and `Piece.of(letter)`, a small factory, which is the only switch on piece type in the program.

3. "Leaves my king in check": try it and undo. Make the move on the board, ask the board whether your king's square is attacked by the other colour, then put both squares back exactly as they were. "Attacked" reuses `canMove`: can any enemy piece move to that square? No new rules needed.

4. One question for mate and stalemate. "Does the side to move have any legal move?" Try every piece of that colour to every square with the same two checks. No legal move and in check: checkmate. No legal move and not in check: stalemate. Otherwise, if in check, announce it. **NF2:** `move` returns a `MoveResult` record and only the driver turns it into text, so an app could reuse `Game` as it is.

In the interview: write this first. (1) `Square`, `Color`, `Piece` and the six subclasses: 25 minutes; the knight and king are one line each, the pawn takes longest. (2) `Board` with the start position and `isAttacked`. (3) `Game.move` with turn and piece checks. (4) "Leaves king in check". (5) Mate and stalemate. (6) Promotion, BOARD and test positions. The domain code here is about 240 lines; if time is short, stop after step 4 and explain step 5 out loud.

Part B — the code

```
import java.util.*;

enum Color {
    WHITE, BLACK;

    Color other()  { return this == WHITE ? BLACK : WHITE; }
    String label() { return this == WHITE ? "White" : "Black"; }
}

record Square(int file, int rank) {                     // file 0..7 = a..h, rank 0..7 = 1..8
    static Square parse(String s) {
        if (!s.matches("[a-h][1-8]")) throw new IllegalArgumentException("Invalid square " + s);
        return new Square(s.charAt(0) - 'a', s.charAt(1) - '1');
    }

    @Override public String toString() { return "" + (char) ('a' + file) + (rank + 1); }
}

abstract class Piece {
    final Color color;

    Piece(Color color) { this.color = color; }

    abstract String name();
    abstract char letter();                    // upper case; black pieces print in lower case

    // Can this piece go from -> to by its own movement rules? (Check is the game's job.)
    abstract boolean canMove(Board b, Square from, Square to);

    char symbol() { return color == Color.WHITE ? letter() : Character.toLowerCase(letter()); }

    // For rooks, bishops and queens: a straight line with nothing in between.
    static boolean pathClear(Board b, Square from, Square to) {
        int df = Integer.signum(to.file() - from.file());
        int dr = Integer.signum(to.rank() - from.rank());
        int f = from.file() + df, r = from.rank() + dr;
        while (f != to.file() || r != to.rank()) {
            if (b.at(new Square(f, r)) != null) return false;
            f += df;
            r += dr;
        }
        return true;
    }

    static Piece of(char c) {                           // 'N' = white knight, 'n' = black knight
        Color color = Character.isUpperCase(c) ? Color.WHITE : Color.BLACK;
        return switch (Character.toUpperCase(c)) {
            case 'K' -> new King(color);
            case 'Q' -> new Queen(color);
```

```
            case 'R' -> new Rook(color);
            case 'B' -> new Bishop(color);
            case 'N' -> new Knight(color);
            case 'P' -> new Pawn(color);
            default -> throw new IllegalArgumentException("Unknown piece " + c);
        };
    }
}

class King extends Piece {
    King(Color c) { super(c); }
    String name() { return "king"; }
    char letter() { return 'K'; }
    boolean canMove(Board b, Square f, Square t) {
        return Math.max(Math.abs(t.file() - f.file()), Math.abs(t.rank() - f.rank())) == 1;
    }
}

class Queen extends Piece {
    Queen(Color c) { super(c); }
    String name() { return "queen"; }
    char letter() { return 'Q'; }
    boolean canMove(Board b, Square f, Square t) {
        int df = Math.abs(t.file() - f.file()), dr = Math.abs(t.rank() - f.rank());
        return (df == 0 || dr == 0 || df == dr) && pathClear(b, f, t);
    }
}

class Rook extends Piece {
    Rook(Color c) { super(c); }
    String name() { return "rook"; }
    char letter() { return 'R'; }
    boolean canMove(Board b, Square f, Square t) {
        return (f.file() == t.file() || f.rank() == t.rank()) && pathClear(b, f, t);
    }
}

class Bishop extends Piece {
    Bishop(Color c) { super(c); }
    String name() { return "bishop"; }
    char letter() { return 'B'; }
    boolean canMove(Board b, Square f, Square t) {
        return Math.abs(t.file() - f.file()) == Math.abs(t.rank() - f.rank()) && pathClear(b, f, t);
    }
}

class Knight extends Piece {
    Knight(Color c) { super(c); }
    String name() { return "knight"; }
    char letter() { return 'N'; }
    boolean canMove(Board b, Square f, Square t) {
        int df = Math.abs(t.file() - f.file()), dr = Math.abs(t.rank() - f.rank());
        return df * dr == 2;                            // (1, 2) or (2, 1)
    }
}

class Pawn extends Piece {
    Pawn(Color c) { super(c); }
    String name() { return "pawn"; }
    char letter() { return 'P'; }
    boolean canMove(Board b, Square f, Square t) {
        int dir = color == Color.WHITE ? 1 : -1, startRank = color == Color.WHITE ? 1 : 6;
        int df = t.file() - f.file(), dr = t.rank() - f.rank();
        Piece target = b.at(t);
        if (df == 0 && dr == dir) return target == null;                        // one step
        if (df == 0 && dr == 2 * dir && f.rank() == startRank)          // two from the start
            return target == null && b.at(new Square(f.file(), f.rank() + dir)) == null;
        // capture: one step diagonally forward onto an enemy piece
        return Math.abs(df) == 1 && dr == dir && target != null && target.color != color;
    }
}

class Board {
    private final Piece[][] grid = new Piece[8][8];      // [file][rank]

    static Board standard() {
        Board b = new Board();
        String back = "RNBQKBNR";
        for (int f = 0; f < 8; f++) {
            b.put(new Square(f, 0), Piece.of(back.charAt(f)));
            b.put(new Square(f, 1), Piece.of('P'));
            b.put(new Square(f, 6), Piece.of('p'));
            b.put(new Square(f, 7), Piece.of(Character.toLowerCase(back.charAt(f))));
        }
        return b;
    }

    Piece at(Square s)            { return grid[s.file()][s.rank()]; }
    void put(Square s, Piece p)   { grid[s.file()][s.rank()] = p; }

    List<Square> squaresOf(Color c) {
        List<Square> out = new ArrayList<>();
        for (int f = 0; f < 8; f++)
            for (int r = 0; r < 8; r++)
                if (grid[f][r] != null && grid[f][r].color == c) out.add(new Square(f, r));
        return out;
    }

    boolean isAttacked(Square s, Color by) {
        for (Square from : squaresOf(by))
            if (at(from).canMove(this, from, s)) return true;
```

```
        return false;
    }

    List<String> render() {
        List<String> lines = new ArrayList<>();
        for (int r = 7; r >= 0; r--) {
            StringBuilder line = new StringBuilder().append(r + 1);
            for (int f = 0; f < 8; f++)
                line.append(' ').append(grid[f][r] == null ? '.' : grid[f][r].symbol());
            lines.add(line.toString());
        }
        lines.add("  a b c d e f g h");
        return lines;
    }
}

enum Outcome { NONE, CHECK, CHECKMATE, STALEMATE }

record MoveResult(Piece piece, Square from, Square to, Piece captured, boolean promoted,
                  Outcome outcome) {}

class Game {
    private Board board;
    private Color turn = Color.WHITE;
    private boolean over;

    Game(Board board) { this.board = board; }

    Board board() { return board; }

    MoveResult move(Square from, Square to) {
        if (over) throw new IllegalStateException("Game is over");
        Piece piece = board.at(from);
        if (piece == null) throw new IllegalArgumentException("No piece at " + from);
        if (piece.color != turn)
            throw new IllegalStateException("It's " + turn.label() + "'s turn");
        if (!isPseudoLegal(from, to))
            throw new IllegalArgumentException("Illegal move for " + piece.name() + ": "
                    + from + " -> " + to);
        if (leavesKingInCheck(from, to))
            throw new IllegalArgumentException("That move leaves your king in check");

        Piece captured = board.at(to);
        board.put(to, piece);
        board.put(from, null);
        boolean promoted = piece instanceof Pawn && (to.rank() == 7 || to.rank() == 0);
        if (promoted) board.put(to, new Queen(piece.color));          // always a queen, for now

        turn = turn.other();
        boolean check = inCheck(turn);
        Outcome outcome = check ? Outcome.CHECK : Outcome.NONE;
        if (!hasLegalMove(turn)) {
            over = true;
            outcome = check ? Outcome.CHECKMATE : Outcome.STALEMATE;
        }
        return new MoveResult(piece, from, to, captured, promoted, outcome);
    }

    // The piece's own rule, and not landing on your own piece.
    private boolean isPseudoLegal(Square from, Square to) {
        Piece piece = board.at(from), target = board.at(to);
        return !from.equals(to) && (target == null || target.color != piece.color)
                && piece.canMove(board, from, to);
    }

    // Try the move, look at your own king, then put everything back.
    private boolean leavesKingInCheck(Square from, Square to) {
        Piece piece = board.at(from), target = board.at(to);
        board.put(to, piece);
        board.put(from, null);
        boolean bad = inCheck(piece.color);
        board.put(from, piece);
        board.put(to, target);
        return bad;
    }

    private boolean inCheck(Color c) {
        for (Square s : board.squaresOf(c))
            if (board.at(s) instanceof King) return board.isAttacked(s, c.other());
        return false;                                    // no king on the board (test positions)
    }

    private boolean hasLegalMove(Color c) {
        for (Square from : board.squaresOf(c))
            for (int f = 0; f < 8; f++)
                for (int r = 0; r < 8; r++) {
                    Square to = new Square(f, r);
                    if (isPseudoLegal(from, to) && !leavesKingInCheck(from, to)) return true;
                }
        return false;
    }
}

public class Main {
    public static void main(String[] args) {
        Game game = new Game(Board.standard());
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
```

```
            try {
                switch (p[0]) {
                    case "NEW" -> {
                        game = new Game(Board.standard());
                        System.out.println("New game, White to move");
                    }
                    case "EMPTY" -> {
                        game = new Game(new Board());
                        System.out.println("Empty board, White to move");
                    }
                    case "PLACE" -> {
                        Square s = Square.parse(p[1]);
                        Piece piece = Piece.of(p[2].charAt(0));
                        game.board().put(s, piece);
                        System.out.println("Placed " + piece.color.label().toLowerCase() + " "
                                + piece.name() + " on " + s);
                    }
                    case "MOVE" -> {
                        MoveResult m = game.move(Square.parse(p[1]), Square.parse(p[2]));
                        Piece moved = m.piece();
                        String text = moved.color.label() + " " + moved.name() + " " + m.from()
                                + (m.captured() == null ? "-" : "x") + m.to();
                        if (m.captured() != null) text += ", takes " + m.captured().name();
                        if (m.promoted()) text += ", promotes to queen";
                        System.out.println(text);
                        String mover = moved.color.label(), other = moved.color.other().label();
                        switch (m.outcome()) {
                            case CHECK -> System.out.println(other + " is in check");
                            case CHECKMATE -> System.out.println("Checkmate! " + mover + " wins");
                            case STALEMATE -> System.out.println("Stalemate! It's a draw");
                            case NONE -> { }
                        }
                    }
                    case "BOARD" -> game.board().render().forEach(System.out::println);
                    default -> System.out.println("Unknown command: " + p[0]);
                }
            } catch (IllegalArgumentException | IllegalStateException e) {
                System.out.println(e.getMessage());
            }
        }
    }
}
```

How to read it: `isPseudoLegal` is "the piece's rule, and not onto your own piece"; `leavesKingInCheck` adds the king's safety. `hasLegalMove` uses exactly those two, so mate detection can't disagree with move checking.

Part C — model answers

1. Undo. Command: record every move as an object, `Move(from, to, piece, captured, promoted)`, on a stack. Undo pops it, puts the piece back on `from` (a pawn again if it was promoted), puts the captured piece back on `to`, gives the turn back and clears "over". The Memento alternative, a copy of the whole board per move, is simpler but larger; with 64 squares either works.

2. Castling. It needs history the game doesn't keep today: have the king and that rook ever moved? Then: the squares between them are empty, the king isn't in check, and it doesn't pass through or land on an attacked square. En passant needs the last move too. The move list from answer 1 provides both: history becomes part of the game's state.

3. Is the mate check too slow? At most 16 pieces × 64 squares, each with a check test that scans the enemy pieces: a few hundred thousand simple steps, which is milliseconds. That's fine for two humans. A chess engine would generate only each piece's real moves, and use bitboards (64-bit numbers as sets of squares).

4. Online. The server owns the game and validates every move; the phones only display and send moves. NF2 pays off here: the same `Game` runs on the server. Each game is independent, so it gets its own lock, or one thread handling its moves in order. Save the move list after every move so a game survives a restart, push moves to the opponent over WebSockets, and keep the clocks on the server.

5. "A switch would do." "You'd be right for six pieces that never change: one method with a switch is compact. But the switch doesn't stay in one place: movement, the letter, the name, and later a piece's value for an engine would each switch on the type. A variant piece means editing every one of them. With a class per piece, all of a piece's rules live together, and the only switch left is the factory that reads a letter."

Common mistakes

- Forgetting that a move can't leave your *own* king in check, including moving a pinned piece.
- Testing the check on the real board and forgetting to undo the trial move, or restoring only one square.
- Sliding pieces that jump over pieces, because the path check is missing.
- A pawn that captures straight ahead, or moves two squares when the first one is blocked.
- Printing inside `Game`, so the rules can't be reused by another interface.

Score yourself (1 point each)

☐	An abstract Piece with six subclasses	☐	A shared path check for sliding pieces
☐	The pawn's three rules are right	☐	Moves are refused in the stated order
☐	"Leaves king in check" tries and undoes	☐	Mate and stalemate from "any legal move?"
☐	Promotion to a queen	☐	The game returns results; only the driver prints
☐	Your output matches exactly	☐	Part C 1: undo with Command (or Memento)

Back pages

Warm-up answers

W1 — Rate Limiter.

Algorithm	One client's state	Strength or weakness
Fixed window	window start, count	simplest, but up to twice the limit can pass at a window edge
Sliding log	times of allowed requests	exact, but memory grows with the limit
Token bucket	tokens, last refill time	two numbers; allows a burst, then a steady rate

The lock goes in each client's limiter (`allow()` is a check-then-act), so clients don't block each other. `computeIfAbsent` creates exactly one limiter per client even when threads race; get-then-put could create two, each allowing the full limit. The burst: asha got 3 requests at t=10 and 3 more at t=12 under the fixed window.

W2 — BookMyShow. The layout and seat type belong to the **screen**; free, held or booked belongs to the **show**. A hold is one step under the show's lock: check every seat exists and is free, mark them all held, create the booking. States: HELD → CONFIRMED (pay) → CANCELLED (cancel); HELD → RELEASED (payment fails or time runs out). A scheduled job (the clock tick in the test) releases expired holds, because "not paid in 5 minutes" isn't an event.

#	W3 answer	Why
1	**Strategy**	fare rules that differ and change every year.
2	**State**	several operations, each depending on the card's status.
3	**Adapter**	a vendor's interface we can't change, behind our own.
4	**Observer**	reactions to one event, with more to come.
5	**Chain of Responsibility**	up to X this one decides; otherwise pass it on.
6	**Facade**	several steps in order behind one call, used from three places.
7	**Composite**	a whole and its parts answer the same question.
8	**Nothing**: a scheduled job	"not used for a year" isn't an event.
9	**Builder**	required parts plus many optional ones, never changing afterwards.
10	**Proxy** (a guard)	the same interface, with a role check and a log in front.
11	**Nothing**: a record	two fields that never change.
12	**Prototype**	copy a finished object, then change a little.

Both designs right and 10 or more of the 12: on track. If W1 or W2 was shaky, re-do that problem's Part A from PDF 3 before Problem 2 here.

Mistake log

One row per mistake, in your own words. Before starting PDF 5, read this page and the earlier logs once.

Date	Problem	What I did	What's right	Rule to remember

Re-do schedule

Problem	Attempted (time taken)	Day 3: Part A from memory (15 min)	Day 10: Part A + the core class	Day 30: the whole problem, timed	Score /10
Snake and Ladder					
Cache with TTL					
Meeting Room Scheduler					
Chess					

The core class to rebuild on day 10: `Game.roll` · `TtlCache` with `live()` and the eviction rule · `Room.book` with the clash check · the `Piece` classes and `leavesKingInCheck`. PDF 3's day-30 re-dos are probably due around now too.

Next: PDF 5. A Logging Framework, Hotel Booking, a Task Scheduler and Food Delivery. Its warm-up will ask you to re-design the Cache with TTL and the Meeting Room Scheduler from memory, so the day-10 re-dos above are worth doing before you open it.